



MIDDLE - CONNECTOR AS A SERVICE

MCAAS - DES

Database Extractor n Sender

Manual de usuario, configuración, pruebas, build e
instalación

Versión del proyecto: 1.0.0

Plataforma objetivo: Windows x64

Fecha del manual: 14 de septiembre de 2026

Contenido

1. Objetivo y alcance
2. Arquitectura y flujo
3. Funciones soportadas
4. Requisitos pre-lanzamiento
5. Estructura del proyecto
6. Configuración del .env
7. Script SQL dinámico
8. Detección de nuevos y actualizados
9. Modo HISTORICO
10. Errores, reinicio y estado persistente
11. Logs y monitor
12. Gmail API
13. Pruebas y validación
14. Build y empaquetado
15. Instalación en servidor Windows
16. Operación diaria y recuperación
17. Seguridad recomendada
18. Límites y supuestos
19. Referencia rápida de comandos

Idea central: la base origen nunca necesita conocer los destinos. MCAAS - DES lee mediante un SELECT, identifica cambios localmente y replica cada cambio hacia los destinos configurados de forma independiente.

1. Objetivo y alcance

MCAAS - DES unifica dos responsabilidades: extracción y envío. Su propósito es ejecutar una consulta SQL configurable contra una base de datos origen, convertir el resultado en operaciones de escritura seguras y replicarlas hacia una o varias bases de datos externas.

El sistema no está acoplado a CONTPAQi. CONTPAQi es un caso de uso natural porque normalmente expone información en SQL Server, pero el origen puede ser SQL Server, PostgreSQL o MySQL siempre que se proporcionen credenciales válidas y conectividad de red.

1-10

destinos por instalación

300 ms

pausa por defecto entre
destinos

24/7

operación prevista

La sincronización es cercana a tiempo real mediante polling. Por defecto se espera 2 segundos entre ciclos completos, pero el valor es configurable en `POLL_INTERVAL_MS`.

2. Arquitectura y flujo

```

                                BASE ORIGEN
                        SQL Server / PostgreSQL / MySQL
                                |
                                | EXTRACTION_SCRIPT
                                v
SELECT -> Validación -> Hash de clave + Hash de fila -> Estado local por destino
                                |
                        +--> Destino 1 -> INSERT/UPDATE -> espera 300 ms
                        +--> Destino 2 -> INSERT/UPDATE -> espera 300 ms
                        +--> Destino 3 -> INSERT/UPDATE -> espera 300 ms
                        +--> ... hasta Destino 10
```

Componentes

Componente	Responsabilidad
Extractor	Ejecuta el SQL de lectura y obtiene columnas y filas sin requerir recompilación.
Detector de cambios	Genera huellas SHA-256 a partir de las columnas clave y del contenido completo de cada fila.
Estado local	Guarda, por destino, la última huella enviada. Evita que un fallo en un destino marque como enviado a los demás.
Sender	Construye operaciones parametrizadas para cada motor destino.
Failure tracker	Cuenta fallos dentro de una ventana de tiempo y activa la pausa persistente.
Logger	Genera logs diarios de operación, errores y banderas de datos.
Monitor	Muestra los logs en una consola legible, separada del núcleo 24/7.
Task Scheduler	Arranca el núcleo al iniciar Windows y lo reinicia si el proceso termina inesperadamente.

3. Funciones soportadas

- Extracción dinámica mediante un archivo `.sql`.
- Origen SQL Server, PostgreSQL o MySQL.
- Entre 1 y 10 destinos, cada uno con credenciales y política histórica independientes.
- Conexión por URL o por host, puerto, base, usuario y contraseña.
- TLS/SSL y certificado CA opcional para PostgreSQL/MySQL.
- Tabla destino definida en el script, con sobrescritura opcional por destino.
- Detección de filas nuevas y filas modificadas.
- Modo normal: UPDATE por clave; si no existe, INSERT.
- Modo histórico: INSERT de una nueva versión cada vez que cambia una fila.
- Batches transaccionales configurables.
- Logs sin almacenar el contenido de las filas en el archivo de banderas.
- Alertas por Gmail API con OAuth 2.0.
- Comandos de diagnóstico sin levantar el servicio permanentemente.

- Build Windows autocontenido para PC sin Node.js.

No crea tablas automáticamente. Las tablas de destino deben existir antes del lanzamiento y sus columnas deben ser compatibles con el resultado del SELECT.

4. Requisitos pre-lanzamiento

4.1 Base origen

- [] Servidor accesible desde la PC donde correrá MCAAS - DES.
- [] TCP/IP habilitado cuando el motor lo requiera. En SQL Server esto suele ser relevante.
- [] Usuario dedicado de solo lectura.
- [] Permiso SELECT sobre todas las tablas/vistas usadas por el script.
- [] Una combinación de columnas estable y no nula para usar como `MCAAS_KEY_COLUMNS`.

4.2 Bases destino

- [] Tabla destino creada.
- [] Columnas compatibles por nombre y tipo con el SELECT.
- [] Usuario con permisos INSERT y UPDATE en destinos normales.
- [] Usuario con permiso INSERT en destinos históricos.
- [] Reglas de firewall y puertos abiertos desde el servidor MCAAS.
- [] TLS/CA configurado si el proveedor, como Aiven, lo exige.

4.3 Máquina de compilación

Solo la máquina que compila necesita Node.js y npm. Se recomienda Node 22 o superior para el pipeline incluido. El servidor que recibe el build no necesita Node.js.

4.4 Servidor de operación

Windows x64, permisos de Administrador durante la instalación, conectividad a todas las bases y espacio para logs/estado. La aplicación se registra mediante Task Scheduler.

Para un servidor de renta con CONTRAQi: no se requiere instalar herramientas de desarrollo. Compile en otra PC, copie la carpeta de release y ejecute el instalador PowerShell como Administrador.

5. Estructura del proyecto

MCAAS-DES/	
assets/	Icono PNG e ICO
docs/	Manual
installer/	Script Inno Setup
scripts/	SQL, instalación, build y utilidades
src/	Código fuente TypeScript
db/	Adaptadores SQL Server/PostgreSQL/MySQL
tests/	Pruebas unitarias y smoke test
.env.example	Plantilla de configuración
package.json	Middle - Connector as a Service - Database Extractor n Sender
README.md	Resumen técnico
dist/windows/	Salida del build

6. Configuración del .env

El archivo entregado `.env.example` documenta todas las variables. En una instalación Windows, el archivo real se encuentra por defecto en:

```
C:\ProgramData\MCAAS-DES\.env
```

6.1 Origen

Variable	Descripción	Ejemplo
SOURCE_DB_TYPE	mssql, postgres o mysql.	mssql
SOURCE_DB_URL	Cadena URL opcional. Si se usa, sustituye host/usuario/etc.	vacío
SOURCE_DB_HOST	Host o IP.	127.0.0.1
SOURCE_DB_PORT	Puerto del motor.	1433
SOURCE_DB_NAME	Base a consultar.	MiEmpresa
SOURCE_DB_USER	Usuario, preferentemente de lectura.	mcaas_reader
SOURCE_DB_PASSWORD	Contraseña. Nunca debe subirse al repositorio.	***
SOURCE_DB_SSL	Activa TLS para motores que usan esta bandera.	false
SOURCE_DB_SSL_CA_PATH	CA opcional, relativa al .env o absoluta.	./certs/ca.pem
SOURCE_DB_ENCRYPT	Opción relevante para SQL Server.	false
SOURCE_DB_TRUST_SERVER_CERTIFICATE	Permite certificado no verificado en SQL Server. Úselo solo si corresponde.	true

6.2 Extracción y sincronización

Variable	Uso	Default
EXTRACTION_SCRIPT	Ruta del SQL de extracción.	./scripts/extraction.sql
POLL_INTERVAL_MS	Espera entre ciclos completos.	2000
INTER_TARGET_DELAY_MS	Espera después de cada destino, haya éxito o fallo.	300
BATCH_SIZE	Filas máximas por transacción de destino.	250
HEARTBEAT_SECONDS	Frecuencia mínima de log cuando no hay cambios.	300

6.3 Destinos

`DESTINATION_COUNT` acepta valores de 1 a 10. Para cada destino se utiliza el patrón `DEST_N_*`.

Variable	Descripción
DEST_N_NAME	Nombre humano mostrado en logs.
DEST_N_DB_TYPE	mssql, postgres o mysql.

Variable	Descripción
DEST_N_DB_URL	URL opcional de conexión.
DEST_N_DB_HOST / PORT / NAME / USER / PASSWORD	Credenciales separadas cuando no se usa URL.
DEST_N_DB_SSL / SSL_CA_PATH	Parámetros de TLS.
DEST_N_HISTORICO	<code>true</code> para insertar cada versión; <code>false</code> para mantener la fila actual.
DEST_N_TARGET_TABLE	Opcional. Permite usar una tabla diferente a la definida en el script.

6.4 Estado, logs y fallos

Variable	Descripción	Default
STATE_DIR	Huella de sincronización y estado persistente.	<code>./state</code>
LOG_DIR	Directorio de logs.	<code>./logs</code>
LOG_RETENTION_DAYS	Retención automática de logs diarios.	<code>30</code>
DATA_FLAG_PER_ROW	Registra hashes de cambio sin valores.	<code>true</code>
FAILURE_THRESHOLD	Número de fallos para considerar error persistente.	<code>5</code>
FAILURE_WINDOW_HOURS	Ventana temporal para contar los fallos.	<code>1</code>
AUTO_RESTART_DELAY_MS	Pausa antes de reiniciar el ciclo tras un fallo fatal.	<code>5000</code>

7. Script SQL dinámico

El script SQL es el contrato de extracción. Puede sustituirse sin recompilar. Debe devolver un conjunto tabular y declarar la tabla destino y las columnas que identifican de forma única cada fila.

```
-- MCAAS_NAME=Clientes
-- MCAAS_TARGET_TABLE=clientes
-- MCAAS_KEY_COLUMNS=IdCliente,EmpresaId

SELECT
    IdCliente,
    EmpresaId,
    Nombre,
    RFC,
    FechaModificacion
FROM dbo.Clientes;
```

Cómo se convierte el resultado

Si el SELECT devuelve las columnas `IdCliente`, `Nombre` y `FechaModificacion`, el sender genera operaciones equivalentes a:

```
INSERT INTO clientes (IdCliente, Nombre, FechaModificacion)
VALUES (?, ?, ?);
```

Los valores reales se envían como parámetros del driver. Esto evita problemas de comillas, tipos de datos y concatenación insegura.

Clave obligatoria: las columnas indicadas en `MCAAS_KEY_COLUMNS` deben estar presentes en el SELECT, no pueden ser NULL y la combinación debe ser única dentro del resultado.

8. Detección de filas nuevas y actualizadas

Para cada fila se calculan dos huellas:

1. **Key hash:** SHA-256 de las columnas clave. Identifica la entidad sin guardar su valor en el log de banderas.
2. **Data hash:** SHA-256 de toda la fila normalizada. Permite saber si cambió cualquier columna seleccionada.

El estado se guarda por stream y por destino en `state/sync-state.json`. Por eso, si Producción sincroniza y Shadow falla, únicamente Producción actualiza su marca local. Shadow conservará el cambio pendiente para el siguiente intento.

Resultado en destino normal

1. Busca por las columnas clave mediante un UPDATE parametrizado.
2. Si el UPDATE afecta una fila, se considera actualización.
3. Si no existe la clave, ejecuta INSERT.
4. Solo después del commit exitoso se actualiza el estado local de ese destino.

9. Modo HISTORICO

Cuando `DEST_N_HISTORICO=true`, MCAAS - DES conserva la detección de cambios, pero no intenta actualizar la fila existente. Cada versión nueva o modificada genera un INSERT.

Ejemplo: un cliente se extrae a las 10:00. A las 10:15 cambia su nombre. El destino normal queda con el nombre nuevo; el destino histórico recibe una segunda fila con la versión nueva.

La tabla histórica debe estar diseñada para aceptar múltiples filas con las mismas columnas clave de negocio. Es habitual añadir en la propia tabla un ID autonumérico o una marca temporal generada por la base. MCAAS - DES no agrega columnas que no existan en el SELECT.

Semántica de entrega: el modo histórico es de tipo at-least-once. En una caída extremadamente puntual entre el commit remoto y el guardado del estado local, el siguiente arranque podría repetir ese INSERT. Si la ausencia absoluta de duplicados es crítica, diseñe el destino con una clave de idempotencia/versionado.

10. Errores, reinicio y estado persistente

Fallo transitorio

Un error de conexión o escritura se registra y se vuelve a intentar. Los fallos de un destino no impiden que otros destinos ya procesados conserven su estado correcto.

Fallo fatal del ciclo

Si falla la extracción origen o el ciclo principal, el motor desconecta, espera `AUTO_RESTART_DELAY_MS` y vuelve a iniciar conexiones. Además, la tarea de Windows tiene política de reinicio si el proceso termina.

Error persistente

Si se alcanzan `FAILURE_THRESHOLD` fallos dentro de `FAILURE_WINDOW_HOURS`:

1. Se escribe `state/persistent-error.json`.
2. Se muestra el mensaje "Error persistente en curso; requiere atención".
3. Se intenta enviar un correo por Gmail API.
4. La aplicación permanece viva, pero pausa extracción y envío.
5. La pausa continúa aunque se reinicie Windows.

Cómo reanudar

Primero corrija la causa. Después ejecute:

```
mcaas-des.exe --clear-persistent-error --config "C:\ProgramData\MCAAS-DES\.env"
```

El servicio detectará la liberación en su siguiente verificación y reanudará la sincronización.

11. Logs y monitor

Archivo	Contenido
app-AAAA-MM-DD.log	Inicio, heartbeats, extracciones con cambios, envíos y mensajes comprensibles.
error-AAAA-MM-DD.log	Solo eventos de nivel ERROR.
data-flags-AAAA-MM-DD.ndjson	Banderas por cambio con hashes truncados, destino y tipo NEW/UPDATED. No incluye valores de la fila.

El monitor abre una consola que hace tail del log de aplicación. Cerrar la consola no cierra el núcleo: el core corre como una tarea independiente.

```
mcaas-des.exe --monitor --config "C:\ProgramData\MCAAS-DES\.env"
```

Mensajes típicos:

```
[INFO] Produccion: 4 cambio(s) detectado(s) (3 nuevo(s), 1 actualizado(s)). Envío iniciado.  
[SUCCESS] Produccion: sincronización completada. Insertados: 3; actualizados: 1.  
[ERROR] Error persistente en curso; requiere atención. La sincronización quedó pausada.  
[INFO] Enviando correo de alerta mediante Gmail API...
```

12. Activación de Gmail API

MCAAS - DES usa OAuth 2.0 y el scope `https://www.googleapis.com/auth/gmail.send`. No necesita leer la bandeja.

12.1 Google Cloud

1. Cree o seleccione un proyecto en Google Cloud Console.
2. Abra APIs y servicios y habilite **Gmail API**.
3. En Google Auth Platform configure **Branding**, **Audience** y **Data Access**.
4. Si pertenece a Google Workspace y el proyecto será solo interno, use audiencia Internal cuando esté disponible. Si es External, agregue la cuenta emisora como usuario de prueba mientras desarrolla.
5. Cree un OAuth Client ID de tipo **Web application**.
6. Agregue como redirect URI autorizado: `http://localhost:53682/oauth2callback`.

12.2 Obtener refresh token

En una PC de desarrollo con Node instalado, copie `.env.example` a `.env` y cargue:

```
GMAIL_CLIENT_ID=...  
GMAIL_CLIENT_SECRET=...  
GMAIL_REDIRECT_URI=http://localhost:53682/oauth2callback
```

Luego ejecute:

```
node scripts/generate-gmail-token.mjs --config .env
```

Abra la URL mostrada, autorice la cuenta que enviará los avisos y copie el refresh token a:

```
GMAIL_ENABLED=true  
GMAIL_REFRESH_TOKEN=...  
GMAIL_FROM=alertas@tu-dominio.com  
GMAIL_ALERT_TO=soporte@tu-dominio.com
```

12.3 Producción

Importante: Google documenta que, para apps External en estado Testing, los refresh tokens pueden caducar después de 7 días. Para una operación 24/7, use un proyecto/estado de publicación apropiado para producción o una configuración Internal de Workspace cuando corresponda.

Referencias oficiales consultadas: Google Workspace Gmail API - Node.js quickstart, server-side authorization y sending messages; Google OAuth production readiness. Las políticas de OAuth pueden cambiar, por lo que deben revisarse antes de un despliegue productivo.

13. Pruebas y validación

13.1 Sin tocar bases

```
npm run typecheck  
npm test  
npm run validate
```

`validate` revisa el `.env`, existencia del script y metadatos sin abrir conexiones.

13.2 Conectividad

```
npm run check:connections
```

Prueba origen y todos los destinos. No realiza sincronización.

13.3 Una sola ejecución

```
npm run run:once
```

Ejecuta exactamente un ciclo completo. Es el comando recomendado antes de levantar el modo 24/7.

13.4 Con el ejecutable compilado

```
mcaas-des.exe --validate-config --config C:\ruta\.env  
mcaas-des.exe --check-connections --config C:\ruta\.env  
mcaas-des.exe --run-once --config C:\ruta\.env  
mcaas-des.exe --status --config C:\ruta\.env
```

Prueba controlada recomendada: comience con una tabla destino de prueba, `DESTINATION_COUNT=1` y un `SELECT` pequeño. Verifique los datos y después agregue Producción/Shadow/Dev.

14. Build y empaquetado

14.1 Instalar dependencias

```
npm install
```

14.2 Build Windows autocontenido

```
npm run build:win
```

El comando ejecuta typecheck, pruebas, bundle con NCC, crea un ejecutable Windows x64 con `@yao-pkg/pkg` e intenta aplicar el icono/metadata MCAAS - DES con `resedit-cli`.

14.3 ZIP de release

```
npm run package:release
```

Genera una carpeta/ZIP para copiar a la PC servidor. El servidor no necesita Node.js.

14.4 Instalador Setup.exe opcional

En una máquina Windows con Inno Setup 6 instalado:

```
npm run installer:win
```

Salida esperada:

```
dist\windows\mcaas-des.exe
dist\windows\MCAAS-DES-Windows-x64\
dist\windows\MCAAS-DES-Windows-x64.zip
dist\windows\MCAAS-DES-Setup-1.0.0.exe (opcional)
```

15. Instalación en un servidor Windows de CONTPAQi

Opción A: carpeta de release + PowerShell

1. Compile MCAAS - DES en una PC de desarrollo.
2. Copie `MCAAS-DES-Windows-x64.zip` al servidor.
3. Extraiga la carpeta en una ubicación temporal.
4. Abra PowerShell como Administrador.
5. Ejecute:

```
powershell -ExecutionPolicy Bypass -File .\install.ps1
```

El instalador copia el binario a `C:\Program Files\MCAAS-DES` y crea la configuración/estado en `C:\ProgramData\MCAAS-DES`.

Configurar antes del arranque

1. Edite `C:\ProgramData\MCAAS-DES\.env`.
2. Edite `C:\ProgramData\MCAAS-DES\scripts\extraction.sql`.
3. Use un usuario de lectura para la base CONTPAQi.
4. Configure 1 a 10 destinos.
5. Si Aiven exige TLS, configure SSL y, cuando aplique, el archivo CA.

Validar

```
"C:\Program Files\MCAAS-DES\mcaas-des.exe" --validate-config --config "C:\ProgramData\MCAAS-DES\.env"
"C:\Program Files\MCAAS-DES\mcaas-des.exe" --check-connections --config "C:\ProgramData\MCAAS-DES\.env"
"C:\Program Files\MCAAS-DES\mcaas-des.exe" --run-once --config "C:\ProgramData\MCAAS-DES\.env"
```

Activar 24/7

Ejecute `Start-MCAAS-DES.bat` como Administrador o inicie la tarea **MCAAS-DES Core** desde Task Scheduler. La tarea ya está configurada para arrancar con Windows.

Terminal: al iniciar sesión se abre el monitor. Si alguien lo cierra, el core permanece ejecutándose en segundo plano. En un servidor sin sesión interactiva el core sigue funcionando, aunque naturalmente no habrá una ventana visible hasta que un usuario inicie sesión.

Opción B: Setup.exe

Si compiló el instalador Inno Setup, ejecute `MCAAS-DES-Setup-1.0.0.exe` como Administrador y complete los mismos pasos de configuración/validación.

16. Operación diaria y recuperación

Comprobaciones periódicas

[] El log diario contiene heartbeat reciente.

- [] No existe error persistente activo.
- [] El espacio libre en disco es suficiente.
- [] Los destinos reciben filas nuevas/modificadas.
- [] El refresh token de Gmail sigue válido.

Después de cambiar el SQL

Ejecute `--validate-config`, luego `--run-once`. Si cambia la tabla destino o las columnas clave, MCAAS - DES crea un stream de estado distinto, por lo que puede producir una resincronización.

Después de cambiar credenciales

Guarde el `.env`, pruebe `--check-connections` y, si había un error persistente, ejecute `--clear-persistent-error`.

RespalDOS recomendados

- `.env`, protegido como secreto.
- `scripts/extraction.sql`.
- `state/`, si desea conservar exactamente las huellas ya enviadas.

17. Seguridad recomendada

- Use un usuario de solo lectura en la base origen.
- Use usuarios de escritura limitados a las tablas necesarias en destinos.
- No reutilice cuentas administrativas de SQL Server/Aiven.
- Proteja `C:\ProgramData\MCAAS-DES\.env` con permisos NTFS restringidos.
- No incluya el `.env` real en Git, ZIP de soporte ni correo.
- Use TLS para bases remotas.
- Evite `SSL_REJECT_UNAUTHORIZED=false` en producción salvo que el proveedor/arquitectura lo requiera y el riesgo haya sido aceptado.
- Rote contraseñas y refresh tokens cuando cambie el personal responsable.
- Los logs de banderas no contienen valores, pero siguen siendo metadatos operativos; protéjalos.

18. Límites y supuestos

Tema	Comportamiento actual
Tiempo real	Polling configurable, no CDC/binlog nativo. El mínimo práctico depende del tamaño del SELECT y de la red.
Deletes	No propaga eliminaciones de la base origen.
DDL	No crea ni migra tablas destino.
Esquemas distintos	Puede sobrescribir la tabla por destino con <code>DEST_N_TARGET_TABLE</code> , pero las columnas deben corresponder.
Claves duplicadas	Se consideran un error de extracción porque impiden identificar una fila de forma estable.
Histórico	At-least-once; una caída en la ventana commit remoto/estado local puede producir duplicado.
Volumen	El estado local es JSON. Para datasets de millones de filas o cargas masivas continuas conviene evolucionar el estado a SQLite/LevelDB o usar watermark/CDC.
Cambios de schema	Si el SELECT agrega/quita columnas, los hashes cambian y los destinos deben aceptar el nuevo schema.

19. Referencia rápida de comandos

Desarrollo

```
npm install
npm run typecheck
npm test
npm run validate
npm run check:connections
npm run run:once
npm run dev
```

Build

```
npm run build:win
npm run package:release
npm run installer:win      # solo Windows + Inno Setup 6
npm run package:zip        # ZIP del proyecto fuente
```

Ejecutable

```
mcaas-des.exe --service --config C:\ruta\.env
mcaas-des.exe --run-once --config C:\ruta\.env
mcaas-des.exe --validate-config --config C:\ruta\.env
mcaas-des.exe --check-connections --config C:\ruta\.env
mcaas-des.exe --status --config C:\ruta\.env
mcaas-des.exe --monitor --config C:\ruta\.env
mcaas-des.exe --clear-persistent-error --config C:\ruta\.env
mcaas-des.exe --version
mcaas-des.exe --help
```

Checklist post-lanzamiento

- [] Primer `--run-once` terminó sin error.
- [] Los destinos contienen los registros esperados.
- [] El log de banderas no muestra información sensible.
- [] La tarea MCAAS-DES Core está Running.
- [] Reinicio de prueba del servidor confirmó autoarranque.
- [] Se probó la alerta Gmail en un ambiente controlado.
- [] Se documentaron responsables y procedimiento para limpiar error persistente.